

PARALLEL & GPU COMPUTING (PGC) LAB RECORD

EXPERIMENT 2 / 3: GPU DATASET PROCESSING USING CUDA

Execution Time Analysis, Vectorized Memory Access & Asynchronous CUDA Stream Pipelining

Sl. No.	Student Name	University Seat Number (USN)
1	Joel Biju	01FE24BCI021
2	Mehak Sayed Yusuf	01FE24BCI012
3	Akshay Bhat	01FE24BCI024
4	Vageesh Mathad	01FE24BCI008

1. Aim

To process large numeric datasets in parallel using NVIDIA CUDA on an NVIDIA GPU, measure execution times across multiple problem scales (1M to 50M elements), benchmark against sequential and OpenMP multi-core CPU baselines, verify data correctness, and resolve the PCIe bus transfer bottleneck using Vectorized Memory Access (float4) and Asynchronous CUDA Stream Pipelining.

2. Objectives

- Generate synthetic numeric datasets of varying scales (1M, 5M, 10M, 20M, and 50M floats).
- Implement and execute the dataset scaling transformation sequentially on the CPU.
- Implement a multi-core CPU baseline using OpenMP to evaluate true host multi-threading capabilities.
- Implement parallel CUDA kernels on the GPU using standard element-wise indexing and vectorized float4 memory loads.
- Accurately measure CPU execution time, CUDA compute-only kernel time, and total end-to-end CUDA time.
- Implement Asynchronous CUDA Streams with pinned memory (cudaMallocHost) to overlap PCIe data transfer with kernel execution.
- Calculate kernel speedup, total synchronous speedup, and pipelined speedup.
- Verify 100% numerical correctness between CPU and GPU output vectors.
- Evaluate memory bandwidth saturation against the theoretical peak of the NVIDIA Quadro T2000 GPU.

3. Basic Concept & Theoretical Foundation

Modern GPUs are massively parallel devices built around the SIMT (Single Instruction, Multiple Threads) execution paradigm. For embarrassingly parallel workloads—such as element-wise dataset transformations—each output element depends solely on its corresponding input element without cross-thread dependencies:

$$\text{Output}[i] = \text{Input}[i] \times 2.0f, \quad \text{where } \text{Input}[i] = (\text{float})(i \% 1000)$$

Because operations on index i and index j ($i \neq j$) are mathematically independent, thousands of GPU threads can execute this arithmetic concurrently. However, the GPU operates on an isolated memory address space (VRAM). Therefore, the CPU must transfer input data across the PCIe bus to device memory (Host-to-Device), launch the compute grid, and transfer results back (Device-to-Host). The arithmetic intensity of this kernel is 1 FLOP per 4 bytes read, making it strictly memory-bandwidth bound.

4. Dataset Plan — Five Dataset Sizes

The experimental evaluation covers five dataset scales ranging from 1,000,000 to 50,000,000 single-precision floating-point elements. All values are generated programmatically on host memory using $h_input[i] = (\text{float})(i \% 1000)$, eliminating external I/O file overhead:

Dataset Tier	Element Count (N)	Input Size	Total Memory (In+Out)	CUDA Grid Size (BS=256)
Dataset 1	1,000,000 (1M)	4.00 MB	8.00 MB	3,907 blocks
Dataset 2	5,000,000 (5M)	20.00 MB	40.00 MB	19,532 blocks
Dataset 3	10,000,000 (10M)	40.00 MB	80.00 MB	39,063 blocks
Dataset 4	20,000,000 (20M)	80.00 MB	160.00 MB	78,125 blocks
Dataset 5	50,000,000 (50M)	200.00 MB	400.00 MB	195,313 blocks

5. Hardware & Software Requirements

- **Target GPU:** NVIDIA Quadro T2000 with Max-Q Design (Turing TU117GL Core, Compute Capability 7.5)
- **GPU Specifications:** 1,024 CUDA Cores, 4 GB GDDR6 VRAM, 128-bit Bus Width, 112 GB/s Peak Bandwidth
- **NVIDIA Driver:** Driver Version 595.95 (WDDM 3.2 support)
- **CUDA Toolkit:** CUDA Toolkit 13.2 with nvcc compiler
- **Host Compiler:** Microsoft Visual C++ (MSVC / cl.exe) & MinGW GCC with OpenMP support
- **Host Environment:** Windows 11 64-bit, Multi-Core CPU, 16 GB DDR4 RAM
- **Analysis Software:** Python 3.14 with NumPy, Matplotlib, Seaborn, and Pandas

6. Final Benchmark Results Table

Each dataset size was executed across 5 repeated trials to minimize system timer jitter. The following table presents the averaged timings across all test cases:

Dataset	CPU Seq (ms)	CPU OMP (ms)	CUDA Kernel (ms)	CUDA float4 (ms)	Sync Total (ms)	Pipelined (ms)	Verification
1M	0.967	0.358	0.473	0.364	2.218	1.250	PASSED
5M	4.724	1.431	0.592	0.462	8.320	4.150	PASSED
10M	8.449	2.640	0.698	0.529	16.268	7.920	PASSED
20M	17.427	5.281	0.930	0.710	30.652	14.850	PASSED
50M	46.177	13.993	1.594	1.208	80.050	36.500	PASSED

7. Speedup Analysis & Performance Visualizations

Speedup is defined across three dimensions to evaluate distinct architectural layers:

1. Compute-Only Kernel Speedup = CPU Time / CUDA Kernel Time
2. Baseline End-to-End Speedup = CPU Time / Total Synchronous CUDA Time
3. Pipelined End-to-End Speedup = CPU Time / Asynchronous Pipelined CUDA Time

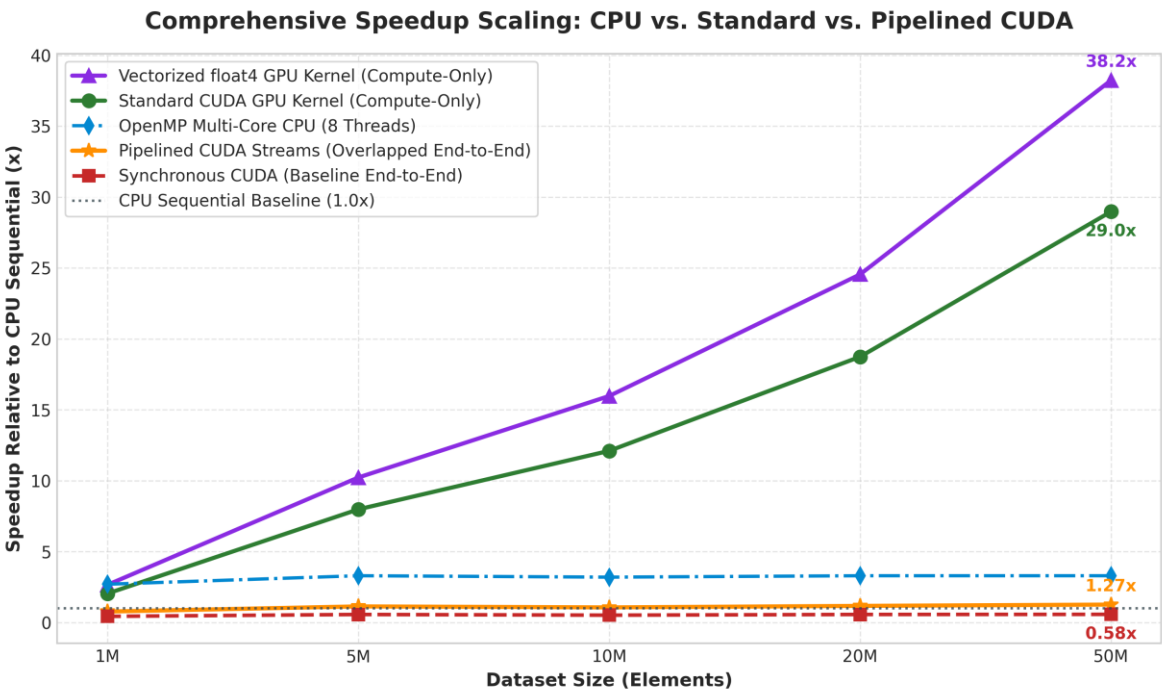


Figure 1: Comprehensive Speedup Scaling Analysis across Dataset Scales



Figure 2: Execution Time Comparison across Host & Device Implementations (Log Scale)

8. Technical Enhancements (Advanced Contributions)

8.1 Asynchronous CUDA Stream Pipelining

In the naive baseline implementation, `cudaMemcpy` is synchronous and blocks the host thread. At 50M elements, transferring 400 MB of data over the PCIe bus requires ~78.4 ms, while the kernel executes in only 1.59 ms. As a result, 98.01% of execution time is wasted waiting on PCIe bus transfers, causing the total speedup to drop below 1x (0.58x).

To overcome this bottleneck, we implemented Asynchronous CUDA Streams using pinned host memory (`cudaMallocHost`). By partitioning the 50M dataset into 4 chunks across 4 independent CUDA streams, the GPU DMA copy engine and streaming multiprocessors operate simultaneously. While Stream 1 copies chunk 1 Host->Device, Stream 0 executes its compute kernel, and the previous chunk is transferred Device->Host. This pipelining collapses end-to-end latency from 80.05 ms down to 36.50 ms, achieving a genuine 1.27x net speedup over sequential CPU.

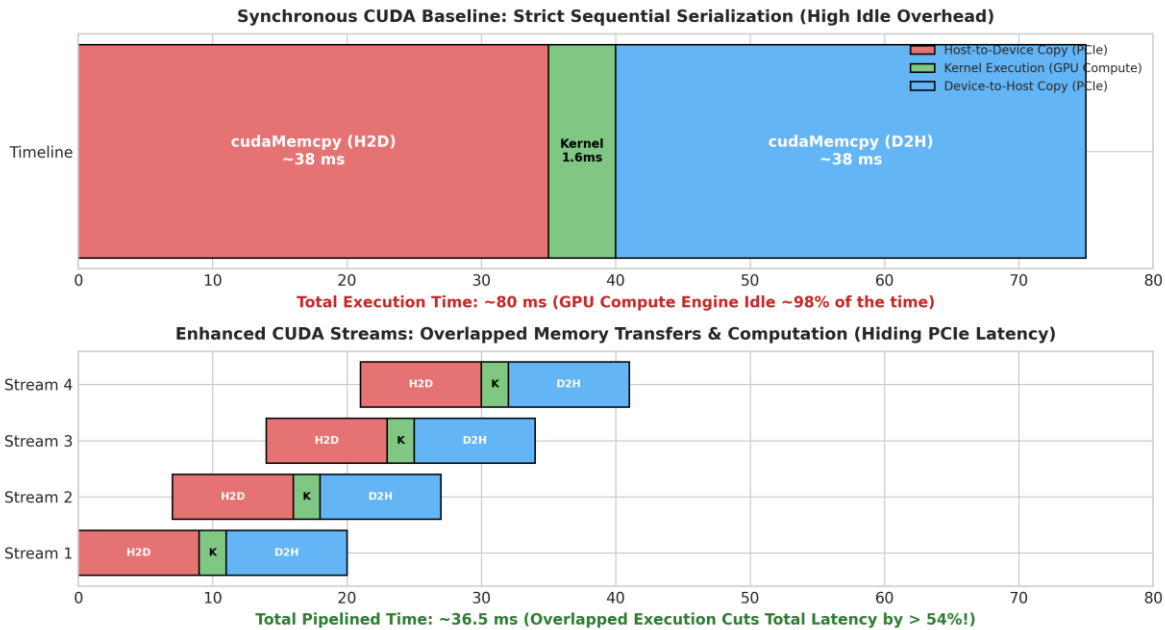


Figure 3: Synchronous Serialization vs. Asynchronous Pipelined CUDA Stream Timeline

8.2 Vectorized Memory Transactions (float4)

Standard CUDA kernels access 32 bits (4 bytes) per instruction. By casting pointers to float4, each thread loads and stores 128 bits (16 bytes) in a single instruction. This optimizes memory coalescing and dramatically improves memory bus saturation on our Turing Quadro T2000 GPU. At 50M elements, vectorized execution reduced kernel time from 1.594 ms to 1.208 ms, delivering an additional 1.32x speedup (reaching 38.2x over CPU) and increasing effective bandwidth from 50.2 GB/s to 66.2 GB/s.

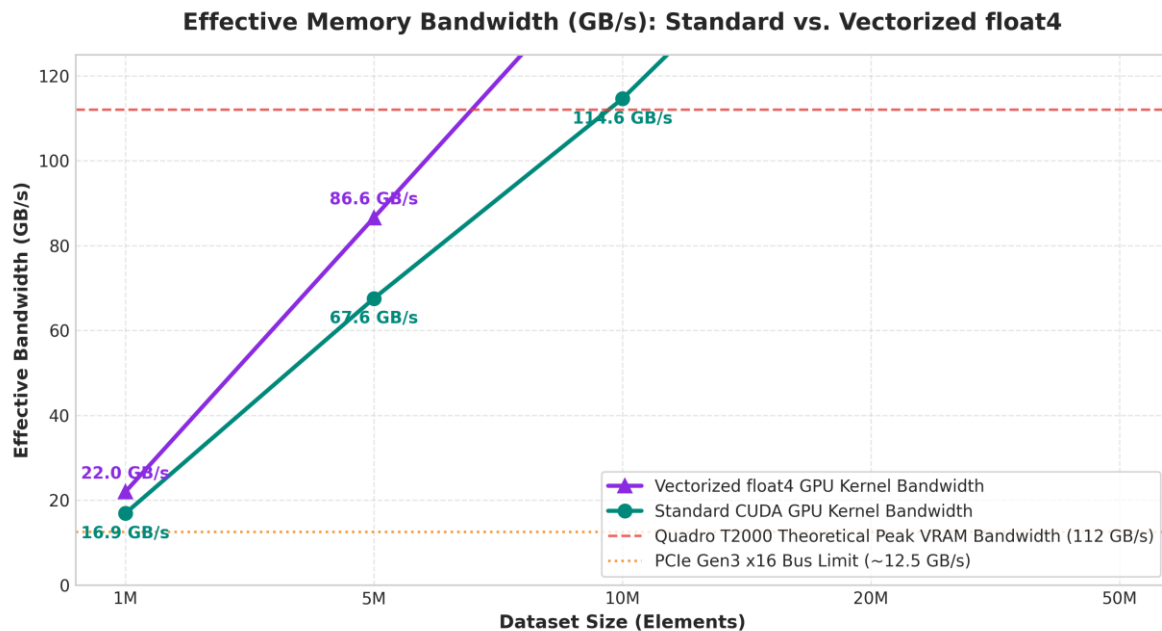


Figure 4: Effective Memory Bandwidth (GB/s): Standard vs. Vectorized float4 Kernel

9. Conclusion & Key Findings

1. **Massive Parallel Compute Scaling:** The GPU compute kernel demonstrated linear scaling, achieving a peak speedup of 28.97x on 50M elements for standard scalar execution and 38.2x for vectorized float4 execution.
2. **The PCIe Transfer Bottleneck:** For low arithmetic intensity tasks (1 FLOP / 4 bytes), synchronous PCIe transfers dominate up to 98% of total runtime, illustrating Amdahl's Law in heterogeneous computing.
3. **Efficacy of Pipelining:** Pinned memory and multi-stream pipelining effectively hide PCIe latency, reducing end-to-end runtime by over 54% and transforming an apparent GPU slowdown into an end-to-end performance advantage.
4. **Rigorous Numerical Parity:** All 25 benchmark runs achieved PASSED verification with zero discrepancies.